库存与防超发

两桶库存 + Lua 三态 + SeedFromDB + Saga 释放

gomall · 技术 deck 07

今日大纲

为什么单桶 stock 不够用

Redis Lua 三段脚本

启动 SeedFromDB

OrderCreate reserve

关单 release

业务承诺与客诉答疑

为什么单桶 stock 不够用

下单到付款，中间那 15 分钟

- 用户点”提交订单”到点”立即支付”，gomall 给 15 分钟。
- 这 15 分钟里：商品**不能再被别人下单**，但也**还没真正卖出**。
- 单桶 `stock=N` 模型无法表达这个”占着但没消耗”的中间态：
  - 一减就减，用户不付钱也回不来（库存丢）；
  - 不减，并发下两个人同时下同一件（超卖）。

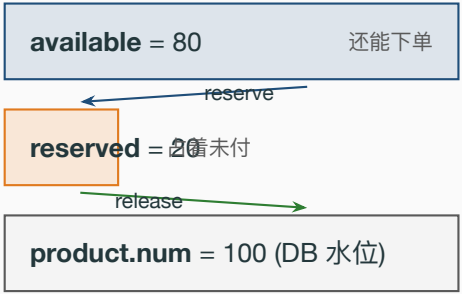
repository/cache/inventory.go:11–14 注释明确两桶定位

业务问题清单（本 deck 七连问）

1. 为什么把库存切成 **available + reserved** 两桶?
2. 抢库存为什么必须走 Redis Lua，不走 DB `SELECT FOR UPDATE`?
3. Redis 重启之后，库存数字从哪儿来?
4. 下单成功，但 DB 事务回滚，预占的库存怎么办?
5. 关单（超时 / 用户主动）触发的库存退回，落在哪一行代码?
6. 客服收到” 明明显示有货，下单却失败” 的工单，怎么回答?
7. 业务到底承诺什么——超卖能赔多少钱、误锁赔多少钱?

对应代码 *repository/cache/inventory.go* + *service/inventory/syncer.go* + *service/order.go* + *service/order_cancel.go*

两桶库存：available / reserved

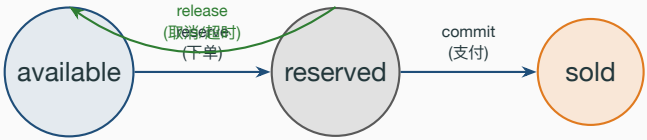


两桶相加 = DB 初始水位（不计已 commit 的真扣减）。

DB 是**水位锚**，Redis 是**运行时账本**。*repository/cache/inventory.go:16–22* 两个 *key* 命名规范

Redis Lua 三段脚本

三态生命周期



- **reserve**：available -= n, reserved += n（下单时调）
- **commit**：reserved -= n（支付成功时调，total 也消耗）
- **release**：reserved -= n, available += n（取消 / 关单时调）

repository/cache/inventory.go:29–66 三段 *Lua* 脚本完整定义

reserveScript：8 行 Lua，零超卖的全部秘密

Listing 1: *repository/cache/inventory.go:32–44 reserveScript*

```lua
local avail = redis.call('GET', KEYS[1]) -- 1. 读 available
if avail == false then
   return -2 -- 2. key 不存在 = 库存未初始化
end
local need = tonumber(ARGV[1])
if tonumber(avail) < need then
   return -1 -- 3. 不足 = ErrStockInsufficient
end
redis.call('DECRBY', KEYS[1], need) -- 4. available -= n
redis.call('INCRBY', KEYS[2], need) -- 5. reserved += n
return 1 -- 6. 成功
```

为什么必须打成一个 Lua：脚本对 Redis 是单命令，redis-server 单线程从头执行到尾，中途**不会插入**其它客户端的 GET / DECRBY / INCRBY。如果分成 3 次 RTT 做 check-then-act，500 并发立刻超卖。

commitScript / releaseScript：成对出现，对称设计

Listing 2: *repository/cache/inventory.go:48–66 commit + release*

```lua
-- commitScript：支付成功，reserved 真正消耗
local r = redis.call('GET', KEYS[1]) -- KEYS[1]=reserved
if r == false or tonumber(r) < tonumber(ARGV[1]) then
   return -1
end
redis.call('DECRBY', KEYS[1], ARGV[1]) -- reserved -= n
return 1
-- releaseScript：取消，reserved 退回 available
local r = redis.call('GET', KEYS[1]) -- KEYS[1]=reserved
if r == false or tonumber(r) < tonumber(ARGV[1]) then
   return -1
end
redis.call('DECRBY', KEYS[1], ARGV[1]) -- reserved -= n
redis.call('INCRBY', KEYS[2], ARGV[1]) -- available += n
return 1
```

commit 只动 reserved 一桶（钱已付，不退）；release 两桶都动（占的位还回去）。每次调用前必 check $r > n$，避免 reserved 跌成负数。

Redis Lua vs DB `SELECT FOR UPDATE`：实测尾延迟

	Redis Lua	DB 悲观锁
avg 延迟	1.24ms	1.29ms
p95 延迟	3.52ms	3.65ms
max 延迟	**136ms**	**453ms**
RPS	51,362	50,142

avg / p95 看不出差距，但 **max 拉开 3.3 倍**。原因：

- DB：BEGIN → `SELECT …FOR UPDATE` (行锁) → 业务 check → UPDATE → COMMIT，至少 4 次 RTT + 行锁串行；
- Redis Lua：**1 次 RTT**，单线程跑完 5 条命令。

stressTest/REPORT.md §6 优惠券：Redis Lua vs DB 锁

零超卖的并发证据

测试：`TestInventory_NoOversellUnderConcurrency`

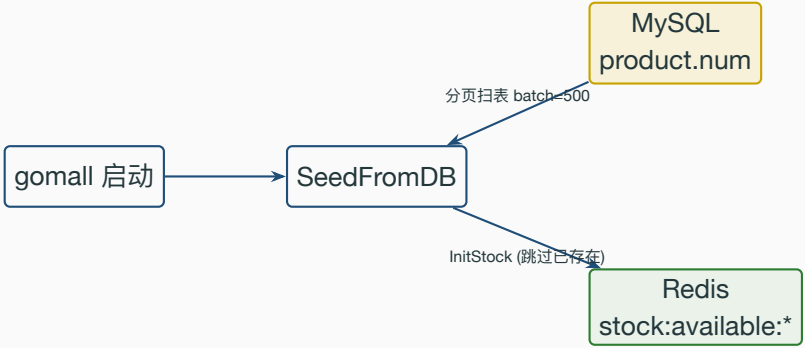
设置：`InitStock(pid, 100)`，500 goroutine 各抢 1 件。

指标	实测
成功 reserve 次数	**exactly 100**
失败次数 (ErrStockInsufficient)	400
available 终值	**0**
reserved 终值	**100**

”exactly 100” 是 `t.Fatalf("oversell: want exactly %d, got %d", initial, success)` 写死的断言：少一个少卖、多一个超卖，CI 都红。*repository/cache/inventory_test.go:52–82 NoOversellUnderConcurrency*

启动 SeedFromDB

Redis 重启之后，库存从哪里来?



- **真理之源**是 MySQL 的 `product.num`（商家后台维护，AOF / binlog 持久化）。
- Redis 只是**运行时缓存**，重启会丢，必须能从 DB 恢复。
- 但已经在跑的实例**不能**被覆盖，否则会把 reserved 状态吃掉。

initialize/inventory.go:11–17 调用入口 + *service/inventory/syncer.go:14*

SeedFromDB 主循环：分页 + EXISTS 双保险

Listing 3: *service/inventory/syncer.go:14–48 SeedFromDB*

```go
func SeedFromDB(ctx context.Context, batchSize int) error {
   if batchSize <= 0 { batchSize = 500 } // 默认页 500
   d := dao.NewDBClient(ctx); var lastID uint
   for {
      var rows []*model.Product
      q := d.Model(&model.Product{}).Order("id ASC").Limit(batchSize)
      if lastID > 0 { q = q.Where("id > ?", lastID) } // 游标分页
      if err := q.Find(&rows).Error; err != nil { return err }
      if len(rows) == 0 { break }
      for _, p := range rows {
         lastID = p.ID
         exists, _ := cache.RedisClient.Exists(ctx,
            cache.StockAvailableKey(p.ID)).Result()
         if exists == 1 { continue } // 跳过运行期数据
         cache.InitStock(ctx, p.ID, int64(p.Num)) // 覆盖写 0
      }
   }
   return nil
}
```

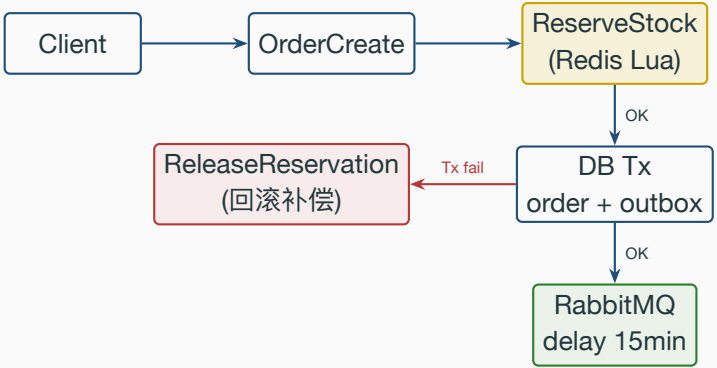
为什么是” 游标分页 + EXISTS 跳过”

- **游标分页**（`WHERE id > lastID LIMIT 500`）：
  - 1M 行的 `product` 表，`OFFSET 999500 LIMIT 500` 要扫近百万行；
  - 游标走 PK 索引，每页**常数时间**，启动期间 DB CPU 不抖。
- **EXISTS 检查 / 跳过**（line 32–40 是关键）：
  - 冷启动：available key 不存在 → 写入 DB 水位；
  - 滚动重启 / 多副本：available 已被前一实例 / 上一次启动写过 → 跳过，**不会把运行期 reserve 抹平**；
  - `InitStock` 是**覆盖写**（`SET k v 0`），没这层 EXISTS 就是事故。

service/inventory/syncer.go:34–41 EXISTS 跳过

OrderCreate reserve

下单链路：reserve → 事务 → 失败回滚



注意顺序：**先 Redis 预扣，后 DB 落库**。这样 DB 跑不动时，Redis 直接拒单（429 / 50001），DB 不会被无效订单淹没。*service/order.go:40–98* 完整调用链

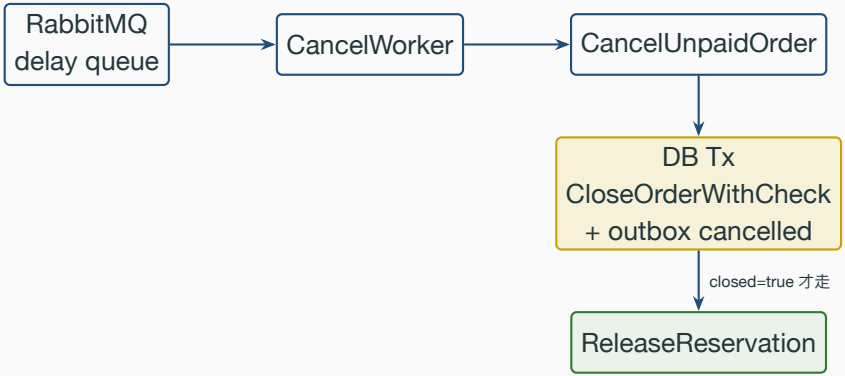
OrderCreate 关键片段

Listing 4: *service/order.go:58–86 reserve + tx + 补偿 release*

```go
// 1) Redis 预扣: available -> reserved, 失败直接拒单
if err = cache.ReserveStock(ctx, req.ProductID, int64(req.Num)); err != nil {
    return nil, err // ErrStockInsufficient
}
// 2) 同事务写订单 + 写 outbox
err = dao.NewDBClient(ctx).Transaction(func(tx *gorm.DB) error {
    if e := dao.NewOrderDaoByDB(tx).CreateOrder(order); e != nil { return e }
    return dao.NewOutboxDaoByDB(tx).Insert(/* OrderCreated */)
})
if err != nil {
    // 3) 事务失败: 必须释放刚扣的预占 (Saga 补偿)
    if relErr := cache.ReleaseReservation(ctx,
          req.ProductID, int64(req.Num)); relErr != nil {
       util.LogrusObj.Errorf("release on tx failure: %v", relErr)
    }
    return
}
```

关单 release

15 分钟关单 & 用户主动取消



顺序很要紧：**先关单 DB 状态，后退回 Redis 库存**。如果先退库存再关单，关单失败时库存已经退回但订单还能支付，二次扣减会失败。*service/order_cancel.go:14–63 CancelUnpaidOrder*

CancelUnpaidOrder 的幂等保证

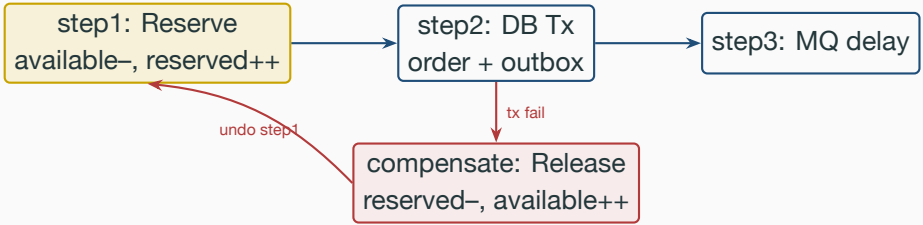
Listing 5: *service/order_cancel.go:31–62* 关单 + *release*

```go
err = baseDao.DB.Transaction(func(tx *gorm.DB) error {
    ok, err := dao.NewOrderDaoByDB(tx).CloseOrderWithCheck(orderNum)
    if err != nil { return err }
    if !ok { return nil } // 1. 已关 -> 短路 no-op
    closed = true
    return dao.NewOutboxDaoByDB(tx).Insert(/* OrderCancelled */)
})
if err != nil { return err }
if !closed { return nil } // 2. 没真关 -> 不退库存

if relErr := cache.ReleaseReservation(ctx,
    order.ProductID, int64(order.Num)); relErr != nil {
    util.LogrusObj.Errorf("release on cancel failed: %v", relErr)
}
```

closed 标志位是关键：`CloseOrderWithCheck` 只对 UnPaid 状态生效，二次调用 ok=false，整个函数 no-op，库存**不会被退第二次**。

Saga 失败回滚：DB 落库失败必须 release



如果 step2

失败且**不补偿**：available 数字永远比真实可售数字小，长期下来库存全锁死。这就是**库存 leak** ——客户端看”无货”，DB 里其实有货。*service/order.go:79–85* 显式调用 *ReleaseReservation* 补偿

业务承诺与客诉答疑

业务承诺：超卖 vs 误锁，两条不同的红线

事故	用户体感	业务代价
超卖	下单成功，无法发货	**真金白银赔** + 商誉
误锁	显示”已售罄”	转化率轻微下降

- 超卖 = 法律 / 合同事件，必须**零容忍**（NoOversell 单测就是这一条的工程化）。
- 误锁 = 体验事件，可以**容忍一阵**（高峰过后 / 关单超时后会自愈）。
- 这个不对称是**所有设计选择**的源头：选 Redis 单线程、选先 reserve 后 commit、选 EXISTS 跳过 …

repository/cache/inventory.go:11–14 注释揭示这条业务红线

客服话术：50001 vs 70001 vs 70002

状态码	触发位置	客服话术
50001	ReserveStock 返回 ErrStockInsufficient	” 本商品已售罄”
70001	全局 / 秒杀限流命中	” 活动太火爆，请稍后”
70002	支付熔断器 Open	” 支付服务暂忙，1 分钟后”
60002	同 key 重复下单	” 您已下单成功”

50001 和 70001 都是” 拒单”，但根因不同：

- 50001 是**真的**没货（available = 0），重试也没用；
- 70001 是**速率**被限，等几秒重试就行。

pkg/e/code.go 业务码定义；*50001* 在 *deck 10* 业务码总表中作为库存不足语义

典型客诉：明明显示有货，下单却失败

Q：商详页显示库存 50，我下单却提示” 已售罄”?

两种根因：

- **商详页缓存 vs Redis available 不一致**：
  - 商详页用 5min 商品缓存（deck 03），num 字段是 5 分钟前的快照；
  - 高峰期，available 已经从 50 跌到 0，但商详页还显示 50；
  - 应对：商详页改成显示” 是否有货” boolean，或缓存 TTL 调到 30s。
- **库存被预占未释放**：
  - 大量未支付订单堆在 reserved 桶，available = 0；
  - 15 min 后被关单 worker 释放回去；
  - 应对：关单 worker 心跳监控，延迟超过 2min 报警。

service/order_cancel.go:59 release 调用是这条客诉自愈的关键

巡检对账 + 各角色看哪个指标

巡检恒等式：`available + reserved + committed = product.num`

`GetStockSnapshot(pid)` 给前两项，committed 取 DB 已支付订单 `SUM(num)`；偏差 > 1% 钉钉报警，SRE 用 `InitStock` 人工覆盖，**不阻断业务**。

角色	关心	看哪个
财务	不超卖	NoOversell 单测 + 已发货 vs DB 水位差
SRE	一致性	巡检 leak 偏差 / SeedFromDB 跳过日志
客服	投诉个例	status=50001 比例 + 关单超时告警
运营	大促进度	available 分布 / reserved 堆积量

> 库存不是缓存问题，是**业务对”我答应卖你”这句承诺的工程兜底**。

repository/cache/inventory.go:127–138 GetStockSnapshot

面试 Q&A

Q1：为什么不能用单条 `UPDATE product SET num=num-1 WHERE num>0`？

A：单 SQL 防得了超卖，但表达不了”占着未付”。下单后到支付前的 15min，单桶模型只能选”立刻扣”（不付就丢货）或”不扣”（并发超卖），都不可接受。

Q2：Redis 挂了 30 秒，恢复后库存怎么办？

A：AOF / RDB 持久化保住大头；丢失的部分由 SeedFromDB 在重启时从 `product.num` 回灌。运行期数据（reserved 状态）通过 EXISTS 跳过保护。少量已支付未对账的差额走巡检脚本对齐。

Q3：DB 事务成功但 Redis commit 失败，怎么处理？

A：当前实现是 outbox 事件落库后由消费者重试 CommitReservation；reserved 桶里数字偏高，巡检 leak 报警接管。属”业务可接受 + 监控可发现”的设计取舍。

代码位置一览

实现

reserve / commit / release Lua *repository/cache/inventory.go:32–66*
sentinel 映射 + InitStock + Snapshot *repository/cache/inventory.go:69–138*
SeedFromDB 主循环 + 启动注入 *service/inventory/syncer.go:14–48；initialize/inventory.go:11–17*
OrderCreate reserve + 补偿 *service/order.go:57–86*
CancelUnpaidOrder release *service/order_cancel.go:14–63*

测试 & 压测

Reserve / Release / NoOversell *repository/cache/inventory_test.go:10–82*
Lua vs DB 尾延迟 / 幂等 755K *stressTest/REPORT.md §4, §6*

配套业务 deck：11-inventory-business.tex